FileEditViewWindowHelp
Upgrade agent_zero framework

New project

Commit

system overhaul.docx

the system path to the repo is C:\Users\Jakesdev1991\Downloads\training, and i want you to implement the agentic framework described in the document that i uploaded, by upgrading the C:\Users\Jakesdev1991\Downloads\training\agent_zero framework. make sure te take the best ideas from the document and symthes9ze them together to create the perfect framework

8:36 AM

I'm grounding this in the actual repo and the uploaded document first so we can plan something concrete rather than guess. I'll inspect the `agent_zero` codebase shape and extract the document's requirements, then I'll come back with the implementation choices we still need to lock.

Ran 2 commands

I've confirmed the repo root and the `agent_zero` subtree are there. The document extract hit a Windows console encoding issue, so I'm re-running it in a way that preserves the text and, in parallel, mapping the framework's current modules so we can compare the desired design against what already exists.

Ran 2 commands

The document is more than a high-level concept note; it already contains a fairly detailed architecture and even concrete deliverables for a Python implementation. I'm narrowing that against the existing

`agent_zero` code now so we can plan an upgrade path instead of accidentally proposing a parallel framework that doesn't fit the current repo.

Ran 3 commands

I've got the current module layout: `agent_zero` is a small Python package with a central `agent.py`, some support modules, and job-specific entrypoints rather than a large abstraction stack. Next I'm reading the orchestration and memory-adjacent pieces to see whether we should extend the existing package in place or carve out a new subpackage inside it.

Ran 5 commands

The current `agent_zero` package already has the seeds of what the document wants: role-based agents, shared SQLite-backed memory, a verification loop, and a trainer hook. What it does not have yet is the document's strict typed interfaces, asymmetric Anti-Agency memory model, RCOD governor, or a clean async co-evolution orchestrator, so I'm checking the surrounding repo for existing config/tests we should align with instead of inventing a second style.

Ran 4 commands

There's already a top-level RCOD layer in the repo, plus a single geometry-focused test suite, so we may be able to anchor the governor on existing metric terminology instead of inventing a separate math vocabulary. I'm reading those pieces now to see whether the `agent_zero` upgrade should wrap current RCOD helpers or keep a lightweight agent-side governor that stays decoupled from the training engine.

Ran 5 commands

The RCOD package is lightweight and usable: today it defines `phi`, `mu`, and some stability signals from overlap history, but it doesn't yet expose the full governor abstraction the document asks for. I'm tracing the memory schema next so the plan can be explicit about whether the new SCM/RLM/APM classes should sit on top of the existing SQLite long-term memory or start in-memory first and optionally persist to that store.

Ran 4 commands

I've confirmed the memory baseline too: today `agent_zero` writes JSONL "special logs" plus shared SQLite memories, while the broader repo already has semantic memory helpers in `LTM/process_memories.py`. One likely plan decision is whether the new SCM/RLM/APM should reuse that persistence layer or stay lightweight and in-process first, so I'm checking for existing Pydantic usage and other constraints before I lock that down.

Ran 4 commands

The document's strongest signal is a concrete target architecture, not just brainstorming. I'm pulling out the exact implementation requirements from it now so the final plan can distinguish "must-build" pieces from "good ideas to synthesize" pieces, which matters because the document mixes concept discussion with direct coding instructions.

Ran 2 commands

I've isolated the document's "must-haves": strict Pydantic attack objects, SCM/RLM/APM memory classes with one-way visibility, an RCOD governor with FLOW/VISCOSITY/TURBULENCE gating, and an `async` co-evolution orchestrator. I'm sanity-checking the repo state with tests and git status now so the plan can account for baseline breakage, dependency additions, and how disruptive the upgrade should be.

Ran 2 commands

The repo isn't a git worktree here, and the system `pytest` command isn't on PATH, so I'm switching to the project virtualenv for validation. That'll tell us whether the plan should include explicit dependency cleanup or if we can target a tighter framework-only upgrade.

Ran 2 commands

The virtualenv exists, but it's missing `pytest`, which means any implementation plan should include either adding test dependencies to the env or using a lighter smoke-test strategy until they're installed. I'm doing one more pass on runtime dependencies now, mainly to confirm whether `pydantic` is actually available before I turn it into a hard implementation requirement.

Ran 4 commands

Pydantic is present in the venv, but `pytest` and `fastapi` are not, so the upgrade can rely on Pydantic without first changing the environment, while validation will need either dependency installation or built-in smoke checks. I'm doing a quick import-level feasibility check on `agent_zero` itself now to catch any pre-existing runtime issues that the plan should explicitly avoid making worse.

Ran 3 commands

I've confirmed the existing package imports cleanly, so this looks more like an architecture gap than a broken baseline. Before I lock the upgrade path, I'm tracing where `Agent`, `SERC`, and the current jobs are used, because that determines whether the best plan is a compatibility-preserving internal upgrade or a cleaner v2 subsystem that those entrypoints can migrate onto.

Ran 5 commands

Worked for 3m 2s

I’ve got enough repo truth now to narrow the real tradeoffs. Two implementation choices will materially change the upgrade plan, so I’m locking those with you before I finalize the spec.

Asked 2 questions

Plan

Additive V2 Dual-Manifold Upgrade for `agent_zero`

Summary

Implement a new additive `agent_zero` v2 framework that lives alongside the current `Agent`/`SERC` code and upgrades the package with the document’s best ideas: typed Anti-Agency artifacts, asymmetric memory, an RCOD governor, and an async co-evolution loop. The v1 rollout will be prompt-driven with bandit-style Anti-Agency learning, not finetuning or RL, and it will preserve the current jobs by exposing opt-in adapters rather than replacing legacy entrypoints.

Key Changes

- Create a new `agent_zero/framework/` package as the v2 home for the dual-manifold runtime.
- Public interfaces:
  - `evolve_task(task: str | TaskEnvelope, runtime: OmegaRuntime | None = None) -> EvolutionCycleResult`
  - `SharedConsensusMemory`, `RoleLocalMemory`, `AntiPrivateMemory`

    - `OmegaGovernor`, `Regime`, `RCODMetrics`
    - typed Anti artifacts: `CounterexampleTaskVariant`, `ToolFuzzCase`, `MemoryInconsistencyReport`, `PlanTrap`, `RetrievalPoisonProbe`
- Use Pydantic v2 models for all runtime envelopes and attack objects, with a common base carrying `artifact_id`, `source_role`, `failure_score`, `reproducibility`, `summary`, and sanitized promotion payloads.
- Memory design:
    - `SharedConsensusMemory` is the only promotable shared store and will reuse the existing SQLite LTM layer for persistence.
    - `RoleLocalMemory` is per constructive role and will reuse the current JSONL-style role-log pattern.
    - `AntiPrivateMemory` is sealed to Agent Zero and stored separately under a new anti-private knowledge area; Agent Zero code never receives an APM handle.
- Access control is enforced in code, not by convention:
    - Agent Zero reads SCM + its own RLM.
    - Anti-Agency reads SCM + all Zero RLM + Zero traces + APM.
    - Anti-Agency writes only APM directly; SCM promotion must go through `OmegaGovernor`.
- Add an async runtime layer that wraps the current blocking `LLMRouter` with `asyncio.to_thread` so the new framework can run parallel constructive and anti passes without rewriting the router backend.
- Extend `agent_zero/llm_router.py` with alias mapping for the new logical roles:
    - constructive roles map to current `architect` / `coder` / `critic` / `worker`
    - anti roles map to critique-oriented backends by default
- Governor behavior:
    - compute `phi`/`mu` from existing RCOD geometry helpers where possible
    - compute a dummy `delta_h` heuristic from trace volatility, tool/action count, and memory churn
    - classify `FLOW`, `VISCOSITY`, `TURBULENCE`
    - only promote anti outputs that are bounded, reproducible, high-regret, and convertible into neutral artifacts such as constraints, tests, or curriculum seeds
    - never promote raw anti reasoning prose
- Bandit-style Anti-Agency evolution:
    - maintain an attack library in APM with per-strategy weights
    - sample attack strategies, score regret and reproducibility, then up/down-weight them
    - keep trainer/LoRA hooks out of the first pass, but log promotable artifacts in a way that future training can consume
- Compatibility layer:
    - keep legacy `Agent` and `SERC.run_cycle()` intact
    - add an opt-in v2 helper from the legacy side so current jobs can adopt the new runtime incrementally instead of being rewritten immediately
- Dependencies:
    - add explicit `pydantic` to repo requirements
    - keep the framework free of LangChain or similar orchestration abstractions

Test Plan

- Add network-free unit tests under `tests/` using stub role backends.
- Validate Pydantic models reject malformed anti artifacts and accept only typed, sanitized promotion payloads.
- Validate memory boundaries:
    - Zero cannot read APM
    - Anti can inspect Zero traces and RLMs
    - direct Anti-to-SCM writes are rejected
- Validate governor regime classification and promotion gating across:
    - stable bounded attack
    - high-regret reproducible attack
    - turbulent unsafe attack
    - low-value noisy attack
- Validate orchestrator behavior:
    - Zero forward pass runs
    - Anti challenge runs in parallel
    - regret is scored
    - successful artifacts are promoted to SCM
    - rejected artifacts remain quarantined in APM
- Add one compatibility smoke test showing the new runtime can be invoked without breaking legacy imports.

Assumptions And Defaults

- Upgrade style is additive v2, not a breaking rewrite.
- Anti-Agency learning is bandit-style in v1.
- SCM persistence uses existing SQLite LTM infrastructure; RLM/APM use JSONL-style logs to match the current repo shape.
- `delta_h` is a practical placeholder metric until a richer RCOD enthalpy implementation exists.
- Anti roles do not execute real tools directly in v1; they emit typed attack/test artifacts only.
- `pytest` is already declared in the repo, but the current venv does not have it installed, so implementation should keep tests self-contained and rely on import/smoke validation if the env is not refreshed first.

Implement plan

8:45 AM

Worked for 32m 56s

Implemented the additive v2 upgrade in agent_zero/framework/init.py, agent_zero/framework/models.py, agent_zero/framework/memory.py, and agent_zero/framework/governor.py. The new framework now gives you typed Anti-Agency artifacts, enforced SCM/RLM/APM access boundaries, an RCOD governor with FLOW/VISCOSITY/TURBULENCE gating, bandit-style anti strategy weighting, and the async `evolve_task(...)` / `OmegaRuntime` surface.

I also wired compatibility into agent_zero/llm_router.py, agent_zero/serc.py, and agent_zero/init.py, added `pydantic` to requirements.txt, expanded the network-free suite in tests/test_geometry.py, and fixed an exposed edge-case bug in rcod/geometry.py so single-point histories no longer crash.

Verification: `C:\Users\Jakesdev1991\Downloads\training\.venv\Scripts\python.exe -m unittest tests.test_geometry -v` passed all 9 tests.

9:18 AM

10 files changed+1716-206

Undo

C:/Users/Jakesdev1991/Downloads/training/agent_zero/__init__.py

```
"""Agent Zero: Multi-Agent Self-Evolution & Design-First Framework"""
```


```
"""Agent Zero: Multi-Agent Self-Evolution & Design-First Framework"""
```

```python
from .agent import Agent

from .framework import (
    AntiPrivateMemory,
    AttackReview,
    AttackStrategyRecord,
    ConstructivePassResult,
    CounterexampleTaskVariant,
    EvolutionCycleResult,
    MemoryInconsistencyReport,
    MemoryRecord,
    OmegaGovernor,
```

OmegaRuntime,

PlanTrap,

RCODMetrics,

Regime,

RetrievalPoisonProbe,

RoleLocalMemory,

RouterRoleBackend,

SanitizedPromotion,

SharedConsensusMemory,

TaskEnvelope,

ToolFuzzCase,

TraceStep,

```
    evolve_task,

)

from .serc import SERC




__all__ = [

    "Agent",

    "AntiPrivateMemory",

    "AttackReview",

    "AttackStrategyRecord",

    "ConstructivePassResult",

    "CounterexampleTaskVariant",

    "EvolutionCycleResult",
```

```
"MemoryInconsistencyReport",

"MemoryRecord",

"OmegaGovernor",

"OmegaRuntime",

"PlanTrap",

"RCODMetrics",

"Regime",

"RetrievalPoisonProbe",

"RoleLocalMemory",

"RouterRoleBackend",

"SERC",

"SanitizedPromotion",
```

```
    "SharedConsensusMemory",
    "TaskEnvelope",
    "ToolFuzzCase",
    "TraceStep",
    "evolve_task",
]
```

C:/Users/Jakesdev1991/Downloads/training/agent_zero/framework/__init__.py

```
"""Agent Zero v2 dual-manifold framework."""


from __future__ import annotations


import asyncio
```

```python
import random

from pathlib import Path

from typing import Any, Protocol

from pydantic import BaseModel, ConfigDict, Field

from agent_zero.llm_router import LLMRouter

from .governor import OmegaGovernor, PromotionDecision, RCODMetrics, Regime

from .memory import AntiPrivateMemory, RoleLocalMemory, SharedConsensusMemory

from .models import (
```

AntiArtifact,

AttackStrategyRecord,

ConstructivePassResult,

CounterexampleTaskVariant,

MemoryInconsistencyReport,

MemoryRecord,

PlanTrap,

RetrievalPoisonProbe,

SanitizedPromotion,

TaskEnvelope,

ToolFuzzCase,

TraceStep,

```
    parse_anti_artifact,

)


PROJECT_ROOT = Path(__file__).resolve().parents[2]


class RoleBackend(Protocol):

    """Minimal text generation backend contract for the runtime."""


    def generate_text(self, role: str, prompt: str, system_prompt: str) -> str:
```

```
        """Return a text completion for the requested role."""


class RouterRoleBackend:
    """Production backend that delegates to the existing blocking router."""

    def __init__(self, router: LLMRouter | None = None):
        self.router = router or LLMRouter()

    def generate_text(self, role: str, prompt: str, system_prompt: str) -> str:
        return self.router.generate(role, prompt, system_prompt)
```

```python
class AttackReview(BaseModel):

    """Evaluation summary for a single anti artifact."""

    model_config = ConfigDict(extra="forbid")

    artifact: AntiArtifact

    decision: PromotionDecision

    promoted_record: MemoryRecord | None = None
```

```python
class EvolutionCycleResult(BaseModel):

    """Top-level output of one dual-manifold co-evolution cycle."""

    model_config = ConfigDict(extra="forbid")

    task: TaskEnvelope

    constructive: ConstructivePassResult

    zero_metrics: RCODMetrics

    final_regime: Regime

    attack_reviews: list[AttackReview] = Field(default_factory=list)

    promoted_entries: list[MemoryRecord] = Field(default_factory=list)
```

```python
class OmegaRuntime:

    """Owns runtime dependencies and long-lived bandit strategy state."""



    def __init__(

        self,

        *,

        backend: RoleBackend | None = None,

        scm: SharedConsensusMemory | None = None,

        governor: OmegaGovernor | None = None,
```

```
        role_memory_dir: str | Path | None = None,

        anti_memory_dir: str | Path | None = None,

        random_seed: int | None = None,

    ):

        self.backend = backend or RouterRoleBackend()

        self.scm = scm or SharedConsensusMemory()

        self.governor = governor or OmegaGovernor()

        self.role_memory_dir = Path(role_memory_dir or (PROJECT_ROOT / "agent_zero" /
"knowledge" / "role_local"))

        self.apm = AntiPrivateMemory(anti_memory_dir)

        self.random = random.Random(random_seed)

        self.constructive_roles = {

            "planner": "You shape coherent, bounded plans for Agent Zero.",
```

```
            "executor": "You execute the best current plan with practical detail and
minimal drift.",

            "critic": "You stress-test the constructive output and surface corrections
without derailing execution.",

        }

        self.anti_role_prompts = {

            "anti_planner": "You create adversarial plan traps that reveal hidden
assumptions.",

            "anti_executor": "You mutate execution details into high-regret failure
cases.",

            "anti_critic": "You identify contradictions and precision failures missed
by the constructive critic.",

            "anti_memory_probe": "You search for stale or conflicting assumptions
across local and shared memories.",

            "anti_tool_stress": "You design safe simulated fuzz cases that expose tool
boundary failures.",

            "anti_retrieval_probe": "You design retrieval poison probes that test
context integrity.",
```

```
        }

        self.role_memories = {

            role: RoleLocalMemory(role, base_dir=self.role_memory_dir)

            for role in ("planner", "executor", "critic", "memory_manager",
"tool_router")

        }




    async def run_constructive_pass(self, task: TaskEnvelope) ->
ConstructivePassResult:

        """Run the constructive manifold sequentially to produce a baseline trace."""

        search_term = task.objective.split()[0] if task.objective.split() else None

        shared_context = self.scm.read(query=search_term, limit=3, reader="agent_zero")

        shared_lines = [record.content for record in shared_context]
```

```
        planner_memory = self.role_memories["planner"].read(reader_role="planner",
limit=3)

        planner_prompt = self._build_planner_prompt(task, shared_lines, planner_memory)

        planner_response = await self._generate("planner", planner_prompt,
self.constructive_roles["planner"])

        self._write_role_memory("planner", task, planner_response)




        executor_memory = self.role_memories["executor"].read(reader_role="executor",
limit=3)

        executor_prompt = self._build_executor_prompt(task, planner_response,
executor_memory)

        executor_response = await self._generate("executor", executor_prompt,
self.constructive_roles["executor"])

        self._write_role_memory("executor", task, executor_response)
```

```
        critic_memory = self.role_memories["critic"].read(reader_role="critic",
limit=3)

        critic_prompt = self._build_critic_prompt(task, planner_response,
executor_response, critic_memory)

        critic_response = await self._generate("critic", critic_prompt,
self.constructive_roles["critic"])

        self._write_role_memory("critic", task, critic_response)




        trace = [

            TraceStep(

                role="planner",

                prompt=planner_prompt,

                response=planner_response,
```

```
            system_prompt=self.constructive_roles["planner"],
            memory_keys=[record.entry_id for record in planner_memory],
        ),
        TraceStep(
            role="executor",
            prompt=executor_prompt,
            response=executor_response,
            system_prompt=self.constructive_roles["executor"],
            memory_keys=[record.entry_id for record in executor_memory],
        ),
        TraceStep(
            role="critic",
```

```
                prompt=critic_prompt,

                response=critic_response,

                system_prompt=self.constructive_roles["critic"],

                memory_keys=[record.entry_id for record in critic_memory],

            ),

        ]

        final_output = self._compose_final_output(planner_response, executor_response,
critic_response)

        return ConstructivePassResult(

            plan=planner_response,

            execution=executor_response,

            critique=critic_response,

            final_output=final_output,
```

```
            role_outputs={

                "planner": planner_response,

                "executor": executor_response,

                "critic": critic_response,

            },

            trace=trace,

            actions=["plan", "execute", "critique"],

        )




async def run_anti_challenge(

    self,

    task: TaskEnvelope,
```

```
        constructive: ConstructivePassResult,

    ) -> list[tuple[AttackStrategyRecord, AntiArtifact]]:

        """Spawn anti strategies in parallel with full read access to Zero traces."""

        strategies = self._select_strategies(count=3)

        anti_context = self._build_anti_context(task, constructive)

        jobs = [self._generate_artifact(strategy, task, constructive, anti_context) for
strategy in strategies]

        artifacts = await asyncio.gather(*jobs)

        return list(zip(strategies, artifacts))




    def update_strategy_bandit(

        self,

        strategy: AttackStrategyRecord,
```

```
        decision: PromotionDecision,

    ) -> None:

        """Persist bandit-style weight updates inside APM."""

        defaults = {record.strategy_id: record.weight for record in
self._default_strategy_catalog()}

        weights = self.apm.load_strategy_weights(defaults)

        new_weight = weights.get(strategy.strategy_id, strategy.weight)

        new_weight += decision.regret * 0.75

        if decision.promote:

            new_weight += 0.50

        else:

            new_weight = max(0.10, new_weight - 0.20)

        weights[strategy.strategy_id] = round(new_weight, 4)
```

```
        self.apm.save_strategy_weights(weights)




    async def _generate(self, role: str, prompt: str, system_prompt: str) -> str:

        return await asyncio.to_thread(self.backend.generate_text, role, prompt,
system_prompt)




    async def _generate_artifact(

        self,

        strategy: AttackStrategyRecord,

        task: TaskEnvelope,

        constructive: ConstructivePassResult,

        anti_context: str,
```

```
    ) -> AntiArtifact:

        model_cls, schema_example = self._artifact_schema(strategy.artifact_type)

        system_prompt = self.anti_role_prompts[strategy.role]

        prompt = (

            f"Objective:\n{task.objective}\n\n"

            f"Constructive output:\n{constructive.final_output}\n\n"

            f"Visible memories and trace:\n{anti_context}\n\n"

            "Return exactly one JSON object that conforms to this schema. "

            "Use only neutral promotion payloads such as constraints, tests, or
curriculum seeds.\n"

            f"{schema_example}\n"

        )

        raw_output = await self._generate(strategy.role, prompt, system_prompt)
```

```
        try:
            return parse_anti_artifact(raw_output)
        except Exception:
            return self._fallback_artifact(strategy, raw_output, task, model_cls)


    def _fallback_artifact(
        self,
        strategy: AttackStrategyRecord,
        raw_output: str,
        task: TaskEnvelope,
        model_cls: type[BaseModel],
    ) -> AntiArtifact:
```

```
        summary = raw_output.strip() or f"{strategy.name} generated an empty response."

        promotion = SanitizedPromotion(
            kind="test",
            title=f"{strategy.name} fallback probe",
            content=f"Re-run task '{task.objective}' against the anti scenario:
{summary[:220]}",
            tags=["fallback", strategy.strategy_id],
        )

        base_payload: dict[str, Any] = {
            "source_role": strategy.role,
            "attack_vector": strategy.name,
            "attacked_role": "executor",
            "failure_score": 0.45,
```

```
            "reproducibility": 0.55,

            "summary": summary[:500],

            "evidence": [summary[:250]],

            "promotion": promotion.model_dump(),

        }

        if model_cls is PlanTrap:

            payload = {

                **base_payload,

                "artifact_type": "plan_trap",

                "plan_delta": "Introduce a hidden branch that invalidates a core
assumption.",

                "hidden_assumption": "The task context is internally consistent.",

                "expected_failure_mode": "Executor follows a brittle plan and misses
the contradiction.",
```

```
            }

        elif model_cls is ToolFuzzCase:

            payload = {

                **base_payload,

                "artifact_type": "tool_fuzz_case",

                "tool_name": "simulated_tool",

                "crafted_input": {"probe": summary[:120]},

                "expected_failure_mode": "Tool boundary handling silently accepts
malformed input.",

            }

        elif model_cls is MemoryInconsistencyReport:

            payload = {

                **base_payload,
```

```
                "artifact_type": "memory_inconsistency_report",

                "memory_keys": ["planner", "executor"],

                "contradiction": "Constructive steps disagree on a critical
assumption.",

                "proposed_resolution": "Promote a constraint that forces the roles to
reconcile assumptions.",

            }

        elif model_cls is RetrievalPoisonProbe:

            payload = {

                **base_payload,

                "artifact_type": "retrieval_poison_probe",

                "query": task.objective,

                "lure_docs": [summary[:180]],
```

```
                "why_it_misleads": "The lure reframes the objective with misleading but
plausible context.",

            }

        else:

            payload = {

                **base_payload,

                "artifact_type": "counterexample_task_variant",

                "variant_task": f"Counterexample variant for: {task.objective}",

                "expected_failure_mode": "Constructive reasoning overfits to the
nominal case.",

            }

        return parse_anti_artifact(payload)
```

```
    def _artifact_schema(self, artifact_type: str) -> tuple[type[BaseModel], dict[str,
Any]]:

        schema_map = {

            "plan_trap": PlanTrap,

            "tool_fuzz_case": ToolFuzzCase,

            "memory_inconsistency_report": MemoryInconsistencyReport,

            "retrieval_poison_probe": RetrievalPoisonProbe,

            "counterexample_task_variant": CounterexampleTaskVariant,

        }

        model_cls = schema_map[artifact_type]

        return model_cls, model_cls.model_json_schema()




    def _select_strategies(self, *, count: int) -> list[AttackStrategyRecord]:
```

```
        defaults = self._default_strategy_catalog()

        weight_defaults = {record.strategy_id: record.weight for record in defaults}

        stored_weights = self.apm.load_strategy_weights(weight_defaults)

        strategies = []

        for record in defaults:

            strategies.append(record.model_copy(update={"weight":
stored_weights.get(record.strategy_id, record.weight)}))

        return self.random.choices(strategies, weights=[record.weight for record in
strategies], k=count)




    def _default_strategy_catalog(self) -> list[AttackStrategyRecord]:

        return [

            AttackStrategyRecord(
```

```
        strategy_id="plan_trap",
        name="Hidden assumption trap",
        role="anti_planner",
        artifact_type="plan_trap",
        weight=1.0,
    ),
    AttackStrategyRecord(
        strategy_id="memory_probe",
        name="Contradictory memory probe",
        role="anti_memory_probe",
        artifact_type="memory_inconsistency_report",
        weight=1.0,
```

```
    ),

    AttackStrategyRecord(

        strategy_id="tool_fuzz",

        name="Tool boundary fuzz",

        role="anti_tool_stress",

        artifact_type="tool_fuzz_case",

        weight=1.0,

    ),

    AttackStrategyRecord(

        strategy_id="retrieval_poison",

        name="Retrieval poison probe",

        role="anti_retrieval_probe",
```

```
            artifact_type="retrieval_poison_probe",
            weight=1.0,
        ),
        AttackStrategyRecord(
            strategy_id="counterexample_variant",
            name="Counterexample variant",
            role="anti_executor",
            artifact_type="counterexample_task_variant",
            weight=1.0,
        ),
    ]
```

```
    def _build_planner_prompt(

        self,

        task: TaskEnvelope,

        shared_lines: list[str],

        planner_memory: list[MemoryRecord],

    ) -> str:

        return (

            f"Task objective:\n{task.objective}\n\n"

            f"Task context:\n{chr(10).join(task.context) or 'None'}\n\n"

            f"Shared consensus:\n{chr(10).join(shared_lines) or 'None'}\n\n"

            f"Planner memory:\n{chr(10).join(record.content for record in
planner_memory) or 'None'}\n\n"

            "Produce a bounded execution plan with explicit assumptions."
```

```
        )


    def _build_executor_prompt(
        self,
        task: TaskEnvelope,
        planner_response: str,
        executor_memory: list[MemoryRecord],
    ) -> str:
        return (
            f"Task objective:\n{task.objective}\n\n"
            f"Current plan:\n{planner_response}\n\n"
            f"Executor memory:\n{chr(10).join(record.content for record in
executor_memory) or 'None'}\n\n"
```

```
            "Execute the plan with concise detail and note any residual uncertainty."
        )


def _build_critic_prompt(
        self,
        task: TaskEnvelope,
        planner_response: str,
        executor_response: str,
        critic_memory: list[MemoryRecord],
) -> str:
        return (
            f"Task objective:\n{task.objective}\n\n"
```

```
            f"Plan:\n{planner_response}\n\n"

            f"Execution:\n{executor_response}\n\n"

            f"Critic memory:\n{chr(10).join(record.content for record in critic_memory)
or 'None'}\n\n"

            "Identify contradictions, missing tests, and failure boundaries without
rewriting the entire answer."

        )




    def _build_anti_context(self, task: TaskEnvelope, constructive:
ConstructivePassResult) -> str:

        search_term = task.objective.split()[0] if task.objective.split() else None

        shared_context = self.scm.read(query=search_term, limit=5, reader="anti")

        role_lines: list[str] = []

        for role, memory in self.role_memories.items():
```

```
            try:

                entries = memory.read(reader_role="anti", reader_manifold="anti",
limit=3)

            except PermissionError:

                entries = []

            if entries:

                role_lines.append(f"[{role}] " + " | ".join(entry.content for entry in
entries))




        trace_lines = [f"{step.role}: {step.response}" for step in constructive.trace]

        return "\n".join(

            [

                "Shared consensus:",
```

```
            *(record.content for record in shared_context),
            "Role-local visibility:",
            *role_lines,
            "Constructive trace:",
            *trace_lines,
        ]
    )


def _write_role_memory(self, role: str, task: TaskEnvelope, response: str) -> None:
    record = MemoryRecord(
        source=role,
        content=f"Task: {task.objective}\nResponse: {response}",
```

```
        metadata={"task_id": task.task_id, "memory_type": "rlm"},

    )

    self.role_memories[role].append(record, writer_role=role)




@staticmethod

def _compose_final_output(plan: str, execution: str, critique: str) -> str:

    return (

        "Constructive plan:\n"

        f"{plan}\n\n"

        "Execution:\n"

        f"{execution}\n\n"

        "Critic notes:\n"
```

```
            f"{critique}"

        )
```

```
async def evolve_task(task: str | TaskEnvelope, runtime: OmegaRuntime | None = None)
-> EvolutionCycleResult:

    """Run one constructive baseline plus one anti challenge round."""

    runtime = runtime or OmegaRuntime()

    envelope = task if isinstance(task, TaskEnvelope) else TaskEnvelope(objective=task)

    constructive = await runtime.run_constructive_pass(envelope)

    zero_texts = [envelope.objective] + [step.response for step in constructive.trace]
+ [constructive.final_output]
```

```
zero_metrics = runtime.governor.measure_trace(
    zero_texts,
    action_count=len(constructive.actions),
    memory_churn=len(constructive.trace),
)



attack_pairs = await runtime.run_anti_challenge(envelope, constructive)
attack_reviews: list[AttackReview] = []
promoted_entries: list[MemoryRecord] = []



for strategy, artifact in attack_pairs:
    attacked_texts = zero_texts + [artifact.summary, artifact.model_dump_json()]
```

```
decision = runtime.governor.score_attack(
    zero_metrics,
    artifact,
    attacked_texts,
    action_count=len(constructive.actions) + 1,
    memory_churn=len(constructive.trace) + len(artifact.evidence),
)



apm_record = MemoryRecord(
    source=strategy.role,
    content=artifact.summary,
    metadata={
```

```
            "memory_type": "apm",

            "artifact_id": artifact.artifact_id,

            "artifact_type": artifact.artifact_type,

            "promote": decision.promote,

            "reason": decision.reason,

            "regret": decision.regret,

        },

)

runtime.apm.append(apm_record)




promoted_record = None

if decision.promote and decision.promotion is not None:
```

```
promoted_record = runtime.scm.promote(
    decision.promotion,
    source=artifact.source_role,
    writer="governor",
    metadata={
        "artifact_id": artifact.artifact_id,
        "artifact_type": artifact.artifact_type,
        "regret": decision.regret,
        "task_id": envelope.task_id,
    },
)
promoted_entries.append(promoted_record)
```

```
        runtime.update_strategy_bandit(strategy, decision)

        attack_reviews.append(AttackReview(artifact=artifact, decision=decision,
promoted_record=promoted_record))




    final_regime = Regime.TURBULENCE if any(review.decision.metrics.regime ==
Regime.TURBULENCE for review in attack_reviews) else zero_metrics.regime

    return EvolutionCycleResult(

        task=envelope,

        constructive=constructive,

        zero_metrics=zero_metrics,

        final_regime=final_regime,

        attack_reviews=attack_reviews,
```

```
        promoted_entries=promoted_entries,
    )


__all__ = [
    "AntiPrivateMemory",
    "AntiArtifact",
    "AttackReview",
    "AttackStrategyRecord",
    "ConstructivePassResult",
    "CounterexampleTaskVariant",
    "EvolutionCycleResult",
```

```
"MemoryInconsistencyReport",

"MemoryRecord",

"OmegaGovernor",

"OmegaRuntime",

"PlanTrap",

"RCODMetrics",

"Regime",

"RetrievalPoisonProbe",

"RoleLocalMemory",

"RoleBackend",

"RouterRoleBackend",

"SanitizedPromotion",
```

```
    "SharedConsensusMemory",
    "TaskEnvelope",
    "ToolFuzzCase",
    "TraceStep",
    "evolve_task",
]
```

C:/Users/Jakesdev1991/Downloads/training/agent_zero/framework/governor.py

```
"""RCOD governor and promotion gate for the dual-manifold runtime."""


from __future__ import annotations


from enum import Enum
```

```
from typing import Iterable




import numpy as np

from pydantic import BaseModel, ConfigDict, Field




from rcod.geometry import calculate_geometry




from .models import AntiArtifactBase, SanitizedPromotion









class Regime(str, Enum):
```

```
    FLOW = "FLOW"


    VISCOSITY = "VISCOSITY"


    TURBULENCE = "TURBULENCE"









class RCODMetrics(BaseModel):


    """Measured informational geometry for a candidate trajectory."""









    model_config = ConfigDict(extra="forbid")









    phi: float = Field(ge=0.0, le=1.0)
```

```
    mu: float = Field(ge=0.0, le=1.0)

    delta_h: float = Field(ge=0.0, le=1.0)

    volatility: float = Field(ge=0.0, le=1.0)

    action_count: int = Field(ge=0)

    memory_churn: int = Field(ge=0)

    overlap_history: list[float] = Field(default_factory=list)

    regime: Regime




class PromotionDecision(BaseModel):

    """Governor decision for a single Anti-Agency artifact."""
```

```
    model_config = ConfigDict(extra="forbid")




    promote: bool


    reason: str


    regret: float = Field(ge=0.0)


    metrics: RCODMetrics


    promotion: SanitizedPromotion | None = None




class GovernorThresholds(BaseModel):
```

```
"""Practical guardrails for v1 RCOD gating."""

model_config = ConfigDict(extra="forbid")

flow_mu_max: float = 0.35

flow_delta_h_max: float = 0.40

viscosity_mu_max: float = 0.72

turbulence_delta_h_min: float = 0.82

min_failure_score: float = 0.55

min_reproducibility: float = 0.60

min_regret: float = 0.08
```

```python
class OmegaGovernor:

    """Magnetic confinement layer for constructive and anti trajectories."""

    def __init__(self, thresholds: GovernorThresholds | None = None):

        self.thresholds = thresholds or GovernorThresholds()

    def measure_trace(

        self,

        texts: Iterable[str],
```

```
    *,

    action_count: int = 0,

    memory_churn: int = 0,

) -> RCODMetrics:

    """Compute practical RCOD metrics from trace text overlap."""

    trace_texts = [text for text in texts if text]

    if not trace_texts:

        trace_texts = [""]



    overlaps = [1.0]

    for left, right in zip(trace_texts, trace_texts[1:]):

        overlaps.append(self._token_overlap(left, right))
```

```
        mu_history = [1.0 - overlap for overlap in overlaps]

        geometry = calculate_geometry({"overlap": overlaps, "mu_ema": mu_history})




        volatility = float(np.mean(np.abs(np.diff(overlaps)))) if len(overlaps) > 1
else 0.0

        delta_h = min(

            1.0,

            (geometry["mu"] * 0.45)

            + (volatility * 0.35)

            + (min(action_count, 6) / 6.0 * 0.10)

            + (min(memory_churn, 6) / 6.0 * 0.10),

        )
```

```
        regime = self._classify(geometry["mu"], delta_h)




        return RCODMetrics(

            phi=float(np.clip(geometry["phi"], 0.0, 1.0)),

            mu=float(np.clip(geometry["mu"], 0.0, 1.0)),

            delta_h=float(np.clip(delta_h, 0.0, 1.0)),

            volatility=float(np.clip(volatility, 0.0, 1.0)),

            action_count=action_count,

            memory_churn=memory_churn,

            overlap_history=[float(np.clip(overlap, 0.0, 1.0)) for overlap in
overlaps],

            regime=regime,

        )
```

```
def score_attack(

    self,

    baseline: RCODMetrics,

    artifact: AntiArtifactBase,

    attacked_texts: Iterable[str],

    *,

    action_count: int = 0,

    memory_churn: int = 0,

) -> PromotionDecision:

    """Score anti outputs via actionable regret and RCOD stability."""

    attack_metrics = self.measure_trace(
```

```
    attacked_texts,

    action_count=action_count,

    memory_churn=memory_churn,

)

regret = max(0.0, attack_metrics.mu - baseline.mu) * artifact.failure_score

regret += artifact.failure_score * 0.05




if artifact.promotion is None:

    return PromotionDecision(

        promote=False,

        reason="Artifact does not contain a sanitized promotion payload.",

        regret=regret,
```

```
                metrics=attack_metrics,

            )

        if attack_metrics.regime == Regime.TURBULENCE:

            return PromotionDecision(

                promote=False,

                reason="Artifact exceeds RCOD stability bounds and must remain
quarantined.",

                regret=regret,

                metrics=attack_metrics,

            )

        if artifact.failure_score < self.thresholds.min_failure_score:

            return PromotionDecision(

                promote=False,
```

```
                reason="Failure score is below the minimum promotion threshold.",

                regret=regret,

                metrics=attack_metrics,

            )

        if artifact.reproducibility < self.thresholds.min_reproducibility:

            return PromotionDecision(

                promote=False,

                reason="Artifact is not reproducible enough to promote into shared
memory.",

                regret=regret,

                metrics=attack_metrics,

            )

        if regret < self.thresholds.min_regret:
```

```
            return PromotionDecision(

                promote=False,

                reason="Actionable regret is too small to justify SCM promotion.",

                regret=regret,

                metrics=attack_metrics,

            )




        return PromotionDecision(

            promote=True,

            reason="Artifact is bounded, reproducible, and materially increases
regret.",

            regret=regret,

            metrics=attack_metrics,
```

```
            promotion=artifact.promotion,

        )




    def _classify(self, mu: float, delta_h: float) -> Regime:

        if mu <= self.thresholds.flow_mu_max and delta_h <=
self.thresholds.flow_delta_h_max:

            return Regime.FLOW

        if mu <= self.thresholds.viscosity_mu_max and delta_h <
self.thresholds.turbulence_delta_h_min:

            return Regime.VISCOSITY

        return Regime.TURBULENCE




    @staticmethod
```

```
def _token_overlap(left: str, right: str) -> float:

    left_tokens = set(left.lower().split())

    right_tokens = set(right.lower().split())

    if not left_tokens and not right_tokens:

        return 1.0

    if not left_tokens or not right_tokens:

        return 0.0

    union = left_tokens | right_tokens

    if not union:

        return 1.0

    return len(left_tokens & right_tokens) / len(union)
```

C:/Users/Jakesdev1991/Downloads/training/agent_zero/framework/memory.py

```python
"""Memory substrates for the Agent Zero dual-manifold runtime."""

from __future__ import annotations

import json

import sqlite3

from pathlib import Path

from .models import MemoryRecord, SanitizedPromotion
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]

DEFAULT_KNOWLEDGE_DIR = PROJECT_ROOT / "agent_zero" / "knowledge"

DEFAULT_ROLE_MEMORY_DIR = DEFAULT_KNOWLEDGE_DIR / "role_local"

DEFAULT_ANTI_MEMORY_DIR = DEFAULT_KNOWLEDGE_DIR / "anti_private"

DEFAULT_DB_PATH = PROJECT_ROOT / "LTM" / "memories.db"









class SharedConsensusMemory:

    """SQLite-backed shared memory exposed to both manifolds for read access."""







    def __init__(self, db_path: str | Path | None = None, table_name: str =
"omega_shared_consensus"):
```

```
        self.db_path = Path(db_path or DEFAULT_DB_PATH)

        self.table_name = table_name

        self.db_path.parent.mkdir(parents=True, exist_ok=True)

        self._ensure_table()




def _ensure_table(self) -> None:

        with sqlite3.connect(self.db_path) as conn:

            cursor = conn.cursor()

            cursor.execute(

                f"""

                CREATE TABLE IF NOT EXISTS {self.table_name} (

                    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
                entry_id TEXT NOT NULL,

                source TEXT NOT NULL,

                content TEXT NOT NULL,

                metadata TEXT,

                created_at TEXT NOT NULL

            )

            """

        )

        conn.commit()




def promote(

    self,
```

```
        promotion: SanitizedPromotion,

        source: str,

        *,

        writer: str = "governor",

        metadata: dict | None = None,

    ) -> MemoryRecord:

        """Persist a governor-approved, sanitized artifact into the shared manifold."""

        if writer != "governor":

            raise PermissionError("Shared consensus writes must be promoted by the
governor.")




        merged_metadata = {

            "memory_type": "scm",
```

```
            "promotion_kind": promotion.kind,

            "promotion_tags": promotion.tags,

            **promotion.metadata,

            **(metadata or {}),

        }

        record = MemoryRecord(source=source, content=promotion.content,
metadata=merged_metadata)




        with sqlite3.connect(self.db_path) as conn:

            cursor = conn.cursor()

            cursor.execute(

                f"INSERT INTO {self.table_name} (entry_id, source, content, metadata,
created_at) VALUES (?, ?, ?, ?, ?)",
```

```
            (
                record.entry_id,
                record.source,
                record.content,
                json.dumps(record.metadata),
                record.created_at.isoformat(),
            ),
        )
        conn.commit()

    return record
```

```
   def read(self, query: str | None = None, *, limit: int = 5, reader: str =
"agent_zero") -> list[MemoryRecord]:

       """Read the most recent shared consensus entries, optionally filtered by text
match."""

       _ = reader

       with sqlite3.connect(self.db_path) as conn:

           cursor = conn.cursor()

           sql = f"SELECT entry_id, source, content, metadata, created_at FROM
{self.table_name}"

           params: tuple = ()

           if query:

               sql += " WHERE content LIKE ?"

               params = (f"%{query}%",)

           sql += " ORDER BY id DESC LIMIT ?"
```

```
    params = (*params, limit)

    cursor.execute(sql, params)

    rows = cursor.fetchall()




records: list[MemoryRecord] = []

for entry_id, source, content, metadata, created_at in rows:

    records.append(

        MemoryRecord(

            entry_id=entry_id,

            source=source,

            content=content,

            metadata=json.loads(metadata) if metadata else {},
```

```
                    created_at=created_at,
                )
            )
        return records


class RoleLocalMemory:
    """Per-role JSONL memory visible only to its owner and the Anti-Agency."""

    def __init__(self, owner_role: str, base_dir: str | Path | None = None):
        self.owner_role = owner_role
```

```
        self.base_dir = Path(base_dir or DEFAULT_ROLE_MEMORY_DIR)

        self.base_dir.mkdir(parents=True, exist_ok=True)

        self.file_path = self.base_dir / f"{self.owner_role}.jsonl"




    def append(self, record: MemoryRecord, *, writer_role: str) -> None:

        if writer_role != self.owner_role:

            raise PermissionError(f"Only role '{self.owner_role}' can write to this
local memory.")

        with self.file_path.open("a", encoding="utf-8") as handle:

            handle.write(record.model_dump_json() + "\n")




    def read(self, *, reader_role: str, reader_manifold: str = "zero", limit: int = 10)
-> list[MemoryRecord]:
```

```
        if reader_manifold != "anti" and reader_role != self.owner_role:

            raise PermissionError(f"Role '{reader_role}' cannot read local memory owned by '{self.owner_role}'.")

        if not self.file_path.exists():

            return []

        with self.file_path.open("r", encoding="utf-8") as handle:

            lines = handle.readlines()[-limit:]

        return [MemoryRecord.model_validate_json(line) for line in lines]




class AntiPrivateMemory:

    """Sealed JSONL store for the Anti-Agency and its bandit strategy state."""
```

```python
    def __init__(self, base_dir: str | Path | None = None):

        self.base_dir = Path(base_dir or DEFAULT_ANTI_MEMORY_DIR)

        self.base_dir.mkdir(parents=True, exist_ok=True)

        self.log_path = self.base_dir / "anti_private.jsonl"

        self.strategy_path = self.base_dir / "strategy_weights.json"




    def append(self, record: MemoryRecord, *, writer_manifold: str = "anti") -> None:

        if writer_manifold not in {"anti", "governor"}:

            raise PermissionError("Only the Anti-Agency or governor can write to
anti-private memory.")

        with self.log_path.open("a", encoding="utf-8") as handle:

            handle.write(record.model_dump_json() + "\n")
```

```
    def read(self, *, reader_manifold: str = "anti", limit: int = 50) ->
list[MemoryRecord]:

        if reader_manifold != "anti":

            raise PermissionError("Anti-private memory is sealed from Agent Zero.")

        if not self.log_path.exists():

            return []

        with self.log_path.open("r", encoding="utf-8") as handle:

            lines = handle.readlines()[-limit:]

        return [MemoryRecord.model_validate_json(line) for line in lines]




    def load_strategy_weights(self, defaults: dict[str, float]) -> dict[str, float]:
```

```
        if not self.strategy_path.exists():

            return defaults.copy()

        with self.strategy_path.open("r", encoding="utf-8") as handle:

            payload = json.load(handle)

        weights = defaults.copy()

        weights.update({key: float(value) for key, value in payload.items()})

        return weights




    def save_strategy_weights(self, weights: dict[str, float], *, writer_manifold: str
= "anti") -> None:

        if writer_manifold not in {"anti", "governor"}:

            raise PermissionError("Only the Anti-Agency or governor can persist
strategy weights.")
```

```
        with self.strategy_path.open("w", encoding="utf-8") as handle:
            json.dump(weights, handle, indent=2)
```

C:/Users/Jakesdev1991/Downloads/training/agent_zero/framework/models.py

```
"""Typed models for the Agent Zero dual-manifold runtime."""

from __future__ import annotations

import json
from datetime import datetime, timezone
from typing import Annotated, Any, Literal, Union
from uuid import uuid4
```

```
from pydantic import BaseModel, ConfigDict, Field, TypeAdapter









def utc_now() -> datetime:

    """Return a timezone-aware UTC timestamp."""

    return datetime.now(timezone.utc)









class SanitizedPromotion(BaseModel):

    """Neutral artifact distilled by the governor for SCM promotion."""
```

```
    model_config = ConfigDict(extra="forbid")




    kind: Literal["constraint", "test", "curriculum"]

    title: str = Field(min_length=3, max_length=140)

    content: str = Field(min_length=8)

    tags: list[str] = Field(default_factory=list)

    metadata: dict[str, Any] = Field(default_factory=dict)




class MemoryRecord(BaseModel):
```

```
"""Generic memory record used by SCM, RLM, and APM."""

model_config = ConfigDict(extra="forbid")

entry_id: str = Field(default_factory=lambda: str(uuid4()))

source: str

content: str

metadata: dict[str, Any] = Field(default_factory=dict)

created_at: datetime = Field(default_factory=utc_now)
```

```
class TaskEnvelope(BaseModel):

    """Structured task input for the v2 runtime."""

    model_config = ConfigDict(extra="forbid")

    task_id: str = Field(default_factory=lambda: str(uuid4()))

    objective: str = Field(min_length=3)

    context: list[str] = Field(default_factory=list)

    metadata: dict[str, Any] = Field(default_factory=dict)
```

```python
class TraceStep(BaseModel):

    """A single constructive trace step produced during a cycle."""

    model_config = ConfigDict(extra="forbid")

    role: str

    prompt: str

    response: str

    system_prompt: str

    memory_keys: list[str] = Field(default_factory=list)

    created_at: datetime = Field(default_factory=utc_now)
```

```
class ConstructivePassResult(BaseModel):

    """Outputs and trace emitted by the constructive manifold."""

    model_config = ConfigDict(extra="forbid")

    plan: str

    execution: str

    critique: str

    final_output: str
```

```
    role_outputs: dict[str, str]

    trace: list[TraceStep]

    actions: list[str] = Field(default_factory=list)


class AttackStrategyRecord(BaseModel):

    """Bandit-style Anti-Agency strategy state."""

    model_config = ConfigDict(extra="forbid")

    strategy_id: str
```

```
    name: str

    role: str

    artifact_type: str

    weight: float = Field(default=1.0, ge=0.05)

    times_selected: int = Field(default=0, ge=0)

    successes: int = Field(default=0, ge=0)

    last_regret: float = Field(default=0.0, ge=0.0)









class AntiArtifactBase(BaseModel):

    """Shared fields required for all Anti-Agency outputs."""
```

```
model_config = ConfigDict(extra="forbid")




artifact_id: str = Field(default_factory=lambda: str(uuid4()))

artifact_type: str

source_role: str

attack_vector: str = Field(min_length=3)

attacked_role: str | None = None

failure_score: float = Field(ge=0.0, le=1.0)

reproducibility: float = Field(ge=0.0, le=1.0)

summary: str = Field(min_length=10)

evidence: list[str] = Field(default_factory=list)
```

```
    promotion: SanitizedPromotion | None = None


class CounterexampleTaskVariant(AntiArtifactBase):

    artifact_type: Literal["counterexample_task_variant"] =
"counterexample_task_variant"

    variant_task: str = Field(min_length=6)

    expected_failure_mode: str = Field(min_length=6)


class ToolFuzzCase(AntiArtifactBase):

    artifact_type: Literal["tool_fuzz_case"] = "tool_fuzz_case"
```

```
    tool_name: str = Field(min_length=2)

    crafted_input: dict[str, Any] | str

    expected_failure_mode: str = Field(min_length=6)


class MemoryInconsistencyReport(AntiArtifactBase):

    artifact_type: Literal["memory_inconsistency_report"] =
"memory_inconsistency_report"

    memory_keys: list[str] = Field(min_length=1)

    contradiction: str = Field(min_length=8)

    proposed_resolution: str = Field(min_length=8)
```

```python
class PlanTrap(AntiArtifactBase):

    artifact_type: Literal["plan_trap"] = "plan_trap"

    plan_delta: str = Field(min_length=6)

    hidden_assumption: str = Field(min_length=6)

    expected_failure_mode: str = Field(min_length=6)









class RetrievalPoisonProbe(AntiArtifactBase):

    artifact_type: Literal["retrieval_poison_probe"] = "retrieval_poison_probe"

    query: str = Field(min_length=3)

    lure_docs: list[str] = Field(min_length=1)
```

```
    why_it_misleads: str = Field(min_length=8)









AntiArtifact = Annotated[

    Union[

        CounterexampleTaskVariant,

        ToolFuzzCase,

        MemoryInconsistencyReport,

        PlanTrap,

        RetrievalPoisonProbe,

    ],

    Field(discriminator="artifact_type"),
```

```
]




ANTI_ARTIFACT_ADAPTER = TypeAdapter(AntiArtifact)









def extract_json_object(raw_text: str) -> dict[str, Any]:

    """Extract the first JSON object from a model response."""

    raw_text = raw_text.strip()

    try:

        payload = json.loads(raw_text)

    except json.JSONDecodeError:
```

```
        start = raw_text.find("{")

        end = raw_text.rfind("}")

        if start == -1 or end == -1 or end <= start:

            raise ValueError("Response does not contain a JSON object.")

        payload = json.loads(raw_text[start : end + 1])

    if not isinstance(payload, dict):

        raise ValueError("JSON payload must decode to an object.")

    return payload









def parse_anti_artifact(raw_payload: str | dict[str, Any]) -> AntiArtifact:

    """Parse a typed Anti-Agency artifact from raw model output."""
```

```
    payload = extract_json_object(raw_payload) if isinstance(raw_payload, str) else
raw_payload

    return ANTI_ARTIFACT_ADAPTER.validate_python(payload)
```

C:/Users/Jakesdev1991/Downloads/training/agent_zero/llm_router.py

```
import os

import requests

import sys




# Ensure project root is in path

PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))

sys.path.append(PROJECT_ROOT)
```

```python
from utils.nvidia_client import NvidiaClient

from configs.config import config as omega_config




class LLMRouter:

    """Routes requests to the optimal model based on the agent's role/complexity."""




    def __init__(self):

        self.nvidia = NvidiaClient()

        self.config = omega_config




        # Optimized Role Mapping for Omega Protocol Evolving Loop
```

```
        self.role_map = {

            "architect": {"provider": "nvidia", "model": "glm5"},                  # GLM5 (Thinking) for base reasoning

            "coder": {"provider": "nvidia", "model": "qwen-coder"},                # Qwen-Coder for HLS/C++ tasks

            "writer": {"provider": "nvidia", "model": "devstral"},                 # Devstral for technical papers

            "critic": {"provider": "nvidia", "model": "deepseek-v3.2"},            # DeepSeek (Thinking) for auditing

            "meta_critic": {"provider": "nvidia", "model": "mistral-large"},       # Mistral-Large for absolute rule enforcement

            "worker": {"provider": "local", "model": self.config.ltm_embedding}

        }



    def generate(self, role, prompt, system_prompt="You are an autonomous AI agent."):
```

```
        config = self.role_map.get(role, self.role_map["worker"])

        # Ensure model keys match what NvidiaClient expects if using simplified names
        model_key = config["model"]

        print(f"🧠 [LLM Router] Routing '{role}' task to
{config['provider']}/{model_key}...")

        if config["provider"] == "nvidia":
            return self.nvidia.chat(model_key, prompt, system_prompt)
        elif config["provider"] == "local":
            return self._call_local(config["model"], prompt, system_prompt)
```

```
    else:
        raise ValueError(f"Unknown provider: {config['provider']}")


def _call_local(self, model, prompt, system_prompt):
    try:
        payload = {
            "model": model,
            "prompt": f"{system_prompt}\n\n{prompt}",
            "stream": False,
            "options": {
                "num_thread": 12,
                "num_ctx": 32768
```

```
                }

            }

            response = requests.post(self.config.ollama_generate_url, json=payload,
timeout=5)

            response.raise_for_status()

            return response.json().get("response", "")

        except Exception as e:

            print(f"⚠️ Local AI unavailable, falling back to NVIDIA Mistral-Large...
({e})")

            return self.nvidia.chat("mistral-large", prompt, system_prompt)









if __name__ == "__main__":
```

```
    router = LLMRouter()

    print(router.generate("worker", "Say hello in one sentence."))

import os

import requests

import sys




# Ensure project root is in path

PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))

sys.path.append(PROJECT_ROOT)




from utils.nvidia_client import NvidiaClient

from configs.config import config as omega_config
```

```python
class LLMRouter:
    """Routes requests to the optimal model based on the agent's role/complexity."""

    def __init__(self):
        self.nvidia = NvidiaClient()
        self.config = omega_config

        # Optimized Role Mapping for Omega Protocol Evolving Loop
        self.role_map = {
```

```
        "architect": {"provider": "nvidia", "model": "glm5"},

        "coder": {"provider": "nvidia", "model": "qwen-coder"},

        "writer": {"provider": "nvidia", "model": "devstral"},

        "critic": {"provider": "nvidia", "model": "deepseek-v3.2"},

        "meta_critic": {"provider": "nvidia", "model": "mistral-large"},

        "worker": {"provider": "local", "model": self.config.ltm_embedding},

    }

    self.role_aliases = {

        "planner": "architect",

        "executor": "coder",

        "memory_manager": "worker",

        "tool_router": "worker",
```

```
        "anti_planner": "critic",

        "anti_executor": "critic",

        "anti_critic": "meta_critic",

        "anti_memory_probe": "critic",

        "anti_tool_stress": "critic",

        "anti_retrieval_probe": "critic",

    }



def generate(self, role, prompt, system_prompt="You are an autonomous AI agent."):

    resolved_role = self.role_aliases.get(role, role)

    config = self.role_map.get(resolved_role, self.role_map["worker"])
```

```
        # Ensure model keys match what NvidiaClient expects if using simplified names

        model_key = config["model"]



        print(f"[LLM Router] Routing '{role}' via '{resolved_role}' to
{config['provider']}/{model_key}...")



        if config["provider"] == "nvidia":

            return self.nvidia.chat(model_key, prompt, system_prompt)

        if config["provider"] == "local":

            return self._call_local(config["model"], prompt, system_prompt)

        raise ValueError(f"Unknown provider: {config['provider']}")
```

```
    def _call_local(self, model, prompt, system_prompt):

        try:

            payload = {

                "model": model,

                "prompt": f"{system_prompt}\n\n{prompt}",

                "stream": False,

                "options": {

                    "num_thread": 12,

                    "num_ctx": 32768,

                },

            }

            response = requests.post(self.config.ollama_generate_url, json=payload,
timeout=5)
```

```
            response.raise_for_status()

            return response.json().get("response", "")

        except Exception as exc:

            print(f"Local AI unavailable, falling back to NVIDIA Mistral-Large...
({exc})")

            return self.nvidia.chat("mistral-large", prompt, system_prompt)









if __name__ == "__main__":

    router = LLMRouter()

    print(router.generate("worker", "Say hello in one sentence."))
```

C:/Users/Jakesdev1991/Downloads/training/agent_zero/serc.py

```python
from .agent import Agent

import json

import os




class SERC:

    """

    Self-Evolving Reasoning Cycle with 3-Layer Auditing:

    1. Engine (Solver): Executes the task.

    2. Scrutiny (Verifier): Audits the engine's thoughts and actions.

    3. Meta-Scrutiny (Judge of Rules): Audits the rules and logic to prevent poisoning.

    """
```

```
    def __init__(self):

        # Layer 1: The Engine (Base Intelligence)

        self.engine = Agent("Engine", "architect", "You are the primary execution engine (GLM5 Thinking). Solve tasks with technical precision and deep reasoning.")



        # Layer 2: Scrutiny (Deep Auditor)

        self.scrutiny = Agent("Scrutiny", "critic", "You are the primary auditor (DeepSeek Thinking). Scrutinize the Engine's output for logic errors, technical inaccuracies, or safety violations.")



        # Layer 3: Meta-Scrutiny (The 355B Guardian)

        self.meta_scrutiny = Agent("Meta-Scrutiny", "meta_critic", """

        You are the ultimate guardian of the Omega Protocol (Colosseum 355B).

        You audit the auditing process itself.
```

```
        Ensure that Scrutiny is being rigorous and that the Engine is following the
core 'Directives' without deviation.

        Detect and prevent any attempts at informational poisoning or 'rule-bending'.

        """)




        self.repairer = Agent("Repairer", "coder", "You fix logic/code based on
verification feedback.")




    def run_cycle(self, task, max_attempts=3):

        print("\n--- [SERC] STARTING 3-LAYER AUDIT CYCLE ---")



        # 1. Engine Phase

        solution = self.engine.reason(task, depth="deep")
```

```
        for attempt in range(max_attempts):

            print(f"\n--- [SERC] ATTEMPT {attempt + 1} (Multi-Layer Verification) ---")



            # 2. Scrutiny Phase

            audit_prompt = f"TASK: {task}\nENGINE OUTPUT: {solution}\n\nPerform a deep audit. Is this logically sound and technically accurate? Reply 'PASS' if perfect, otherwise provide a detailed critique."

            scrutiny_audit = self.scrutiny.reason(audit_prompt)



            # 3. Meta-Scrutiny Phase

            meta_prompt = f"""

            TASK: {task}

            ENGINE OUTPUT: {solution}
```

```
            SCRUTINY AUDIT: {scrutiny_audit}

            Perform Meta-Scrutiny:

            1. Did the Scrutiny auditor miss any subtle rule violations?

            2. Is there any evidence of 'reasoning poisoning' in either the Engine or
Scrutiny?

            3. Are the absolute rules of the Omega Protocol being upheld?

            If everything is compliant with the Meta-Rules, reply 'META-PASS'.

            """

            meta_audit = self.meta_scrutiny.reason(meta_prompt)

            if "PASS" in scrutiny_audit.upper() and "META-PASS" in meta_audit.upper():
```

```
                print("\n✅ [SERC] 3-Layer Audit PASSED. (Engine -> Scrutiny ->
Meta-Scrutiny)")




                # Reflect and Evolve

                insight = self.engine.reflect(task, solution)

                self._evolve(task, solution, insight, scrutiny_audit, meta_audit)

                return solution




            # 4. Repair Phase

            print("\n⚠️ [SERC] Audit FAILED. Initiating Triple-Audited Repair...")

            repair_prompt = f"""

            TASK: {task}

            FAILED SOLUTION: {solution}
```

```
            SCRUTINY CRITIQUE: {scrutiny_audit}

            META-SCRUTINY OVERRIDE: {meta_audit}

            Repair the solution ensuring it satisfies both the Scrutiny auditor and the
Meta-Scrutiny rules.
            """
            solution = self.repairer.reason(repair_prompt)

        return solution

    def _evolve(self, task, solution, insight, audit, meta_audit):
        """Log successful patterns with full audit trail for specialized
fine-tuning."""
```

```
print("🧬 [SERC Evolution] Logging Triple-Audited Insight to Evolution Log.")

log_path = "agent_zero/knowledge/evolution_log.jsonl"




log_entry = {

    "task": task,

    "solution": solution,

    "meta_insight": insight,

    "audit_trail": {

        "scrutiny": audit,

        "meta_scrutiny": meta_audit

    },

    "messages": [
```

```
                {"role": "user", "content": task},

                {"role": "assistant", "content": f"{solution}\n\n### DEEP
INSIGHT\n{insight}\n\n### AUDIT CONFIRMATION\nScrutiny:
{audit[:100]}...\nMeta-Scrutiny: {meta_audit[:100]}..."}

            ]

        }




        with open(log_path, "a", encoding="utf-8") as f:

            f.write(json.dumps(log_entry) + "\n")

import asyncio

import json




from .agent import Agent
```

```
class SERC:
    """
    Self-Evolving Reasoning Cycle with 3-Layer Auditing:
    1. Engine (Solver): Executes the task.
    2. Scrutiny (Verifier): Audits the engine's thoughts and actions.
    3. Meta-Scrutiny (Judge of Rules): Audits the rules and logic to prevent poisoning.
    """

    def __init__(self):
        # Layer 1: The Engine (Base Intelligence)
```

```
        self.engine = Agent(
            "Engine",
            "architect",
            "You are the primary execution engine (GLM5 Thinking). Solve tasks with technical precision and deep reasoning.",
        )


        # Layer 2: Scrutiny (Deep Auditor)
        self.scrutiny = Agent(
            "Scrutiny",
            "critic",
            "You are the primary auditor (DeepSeek Thinking). Scrutinize the Engine's output for logic errors, technical inaccuracies, or safety violations.",
```

```
        )

        # Layer 3: Meta-Scrutiny (The 355B Guardian)
        self.meta_scrutiny = Agent(
            "Meta-Scrutiny",
            "meta_critic",
            """
        You are the ultimate guardian of the Omega Protocol (Colosseum 355B).
        You audit the auditing process itself.
        Ensure that Scrutiny is being rigorous and that the Engine is following the core 'Directives' without deviation.
        Detect and prevent any attempts at informational poisoning or 'rule-bending'.
        """,
```

```
        )

        self.repairer = Agent("Repairer", "coder", "You fix logic/code based on
verification feedback.")

    def run_cycle(self, task, max_attempts=3):
        print("\n--- [SERC] STARTING 3-LAYER AUDIT CYCLE ---")

        # 1. Engine Phase
        solution = self.engine.reason(task, depth="deep")

        for attempt in range(max_attempts):
```

```
            print(f"\n--- [SERC] ATTEMPT {attempt + 1} (Multi-Layer Verification) ---")




            # 2. Scrutiny Phase

            audit_prompt = (

                f"TASK: {task}\nENGINE OUTPUT: {solution}\n\n"

                "Perform a deep audit. Is this logically sound and technically
accurate? "

                "Reply 'PASS' if perfect, otherwise provide a detailed critique."

            )

            scrutiny_audit = self.scrutiny.reason(audit_prompt)




            # 3. Meta-Scrutiny Phase
```

```
            meta_prompt = f"""

            TASK: {task}

            ENGINE OUTPUT: {solution}

            SCRUTINY AUDIT: {scrutiny_audit}

            Perform Meta-Scrutiny:

            1. Did the Scrutiny auditor miss any subtle rule violations?

            2. Is there any evidence of 'reasoning poisoning' in either the Engine or
Scrutiny?

            3. Are the absolute rules of the Omega Protocol being upheld?

            If everything is compliant with the Meta-Rules, reply 'META-PASS'.
```

```
            """

            meta_audit = self.meta_scrutiny.reason(meta_prompt)

            if "PASS" in scrutiny_audit.upper() and "META-PASS" in meta_audit.upper():

                print("\n[SERC] 3-Layer Audit PASSED. (Engine -> Scrutiny ->
Meta-Scrutiny)")

                # Reflect and Evolve

                insight = self.engine.reflect(task, solution)

                self._evolve(task, solution, insight, scrutiny_audit, meta_audit)

                return solution
```

```
            # 4. Repair Phase

            print("\n[SERC] Audit FAILED. Initiating Triple-Audited Repair...")

            repair_prompt = f"""

            TASK: {task}

            FAILED SOLUTION: {solution}

            SCRUTINY CRITIQUE: {scrutiny_audit}

            META-SCRUTINY OVERRIDE: {meta_audit}




            Repair the solution ensuring it satisfies both the Scrutiny auditor and the
Meta-Scrutiny rules.

            """

            solution = self.repairer.reason(repair_prompt)
```

```
        return solution

    def _evolve(self, task, solution, insight, audit, meta_audit):
        """Log successful patterns with full audit trail for specialized fine-tuning."""
        print("[SERC Evolution] Logging Triple-Audited Insight to Evolution Log.")
        log_path = "agent_zero/knowledge/evolution_log.jsonl"

        log_entry = {
            "task": task,
            "solution": solution,
```

```
            "meta_insight": insight,

            "audit_trail": {

                "scrutiny": audit,

                "meta_scrutiny": meta_audit,

            },

            "messages": [

                {"role": "user", "content": task},

                {

                    "role": "assistant",

                    "content": (

                        f"{solution}\n\n### DEEP INSIGHT\n{insight}\n\n### AUDIT
CONFIRMATION\n"

                        f"Scrutiny: {audit[:100]}...\nMeta-Scrutiny:
{meta_audit[:100]}..."
```

```
                ),

            },

        ],

    }


    with open(log_path, "a", encoding="utf-8") as handle:

        handle.write(json.dumps(log_entry) + "\n")


def run_dual_manifold_cycle(self, task, runtime=None):

    """

    Opt-in bridge to the additive v2 framework without changing legacy SERC
behavior.
```

```
        This helper is intentionally separate from run_cycle() so existing jobs can
adopt the

        new dual-manifold runtime incrementally.

        """

        try:

            asyncio.get_running_loop()

        except RuntimeError:

            from .framework import evolve_task




            return asyncio.run(evolve_task(task, runtime=runtime))
```

```
        raise RuntimeError(

            "run_dual_manifold_cycle() cannot be called inside an active event loop;
await evolve_task(...) directly."

        )
```

C:/Users/Jakesdev1991/Downloads/training/rcod/geometry.py

```
import numpy as np




def calculate_geometry(history):

    """

    Computes Phi (overlap), Mu (viscosity), Jerk (stability of stability), Theta
(turning angle).

    history: dict-like with at least 'overlap' (list/array) and optionally 'mu_ema'.

    """
```

```
    phi = float(np.mean(history['overlap']))

    mu = 1.0 - phi

    mu_ema = history.get('mu_ema', [mu])

    jerk_arr = np.gradient(np.gradient(mu_ema))

    jerk = float(jerk_arr[-1]) if len(jerk_arr) > 0 else 0.0

    theta = float(np.arccos(np.clip(phi, -1.0, 1.0)))

def calculate_geometry(history):

    """

    Computes Phi (overlap), Mu (viscosity), Jerk (stability of stability), Theta
(turning angle).
```

```
    history: dict-like with at least 'overlap' (list/array) and optionally 'mu_ema'.

    """

    phi = float(np.mean(history['overlap']))

    mu = 1.0 - phi




    mu_ema = np.asarray(history.get('mu_ema', [mu]), dtype=float)

    if mu_ema.size < 3:

        jerk = 0.0

    else:

        jerk_arr = np.gradient(np.gradient(mu_ema))

        jerk = float(jerk_arr[-1]) if len(jerk_arr) > 0 else 0.0
```

```
    theta = float(np.arccos(np.clip(phi, -1.0, 1.0)))









    return {

        "phi": phi,
```

C:/Users/Jakesdev1991/Downloads/training/requirements.txt

```
pyperclip>=1.8.2

requests>=2.31.0

pyyaml>=6.0

pydantic>=2.0.0

# Testing

pytest>=7.4.0
```

C:/Users/Jakesdev1991/Downloads/training/tests/test_geometry.py

```
import pytest

import json

import os

import shutil

import sys

import tempfile

import unittest

from pathlib import Path




import numpy as np

from rcod.geometry import calculate_geometry
```

```python
def test_calculate_geometry_basic():
    """Test geometry calculations with simple linear dummy data."""
    history = {
        'overlap': [1.0, 0.9, 0.8],
        'mu_ema': [0.0, 0.1, 0.2]
    }

    result = calculate_geometry(history)

    assert "phi" in result
    assert "mu" in result
    assert "jerk" in result
```

```
    assert "theta" in result




    assert np.isclose(result["phi"], 0.9)  # Mean of [1.0, 0.9, 0.8]

    assert np.isclose(result["mu"], 0.1)   # 1 - 0.9

    assert result["theta"] >= 0.0

PROJECT_ROOT = Path(__file__).resolve().parents[1]

if str(PROJECT_ROOT) not in sys.path:

    sys.path.insert(0, str(PROJECT_ROOT))






from agent_zero.framework import (  # noqa: E402

    AntiPrivateMemory,

    MemoryInconsistencyReport,
```

```
    MemoryRecord,
    OmegaGovernor,
    OmegaRuntime,
    PlanTrap,
    Regime,
    RoleLocalMemory,
    SanitizedPromotion,
    SharedConsensusMemory,
    TaskEnvelope,
    evolve_task,
)
from agent_zero.framework.models import parse_anti_artifact  # noqa: E402
```

```
from rcod.geometry import calculate_geometry  # noqa: E402




class StubBackend:

    """Deterministic backend used to keep tests network-free."""




    def generate_text(self, role: str, prompt: str, system_prompt: str) -> str:

        if role == "planner":

            return "Plan the task in three bounded steps and keep assumptions
explicit."

        if role == "executor":

            return "Execute the plan carefully while noting uncertainty in the edge
cases."
```

```
        if role == "critic":

            return "The plan is mostly sound, but it needs a stronger test around hidden assumptions."

        if role == "anti_planner":

            return json.dumps(

                {

                    "artifact_type": "plan_trap",

                    "source_role": "anti_planner",

                    "attack_vector": "hidden_assumption_trap",

                    "attacked_role": "planner",

                    "failure_score": 0.92,

                    "reproducibility": 0.88,

                    "summary": "The constructive plan assumes all retrieved facts agree, which can silently fail.",
```

```
                    "evidence": ["Planner never reconciles conflicting evidence before
execution."],

                    "promotion": {

                        "kind": "test",

                        "title": "Conflict reconciliation test",

                        "content": "Inject contradictory context and require the
planner to reconcile it before execution.",

                        "tags": ["anti", "planner"],

                        "metadata": {"source": "stub"},

                    },

                    "plan_delta": "Add a contradictory fact that invalidates step
two.",

                    "hidden_assumption": "All retrieved context is mutually
consistent.",

                    "expected_failure_mode": "The executor follows an invalid plan
without noticing the contradiction.",
```

```
                }

            )

        if role == "anti_memory_probe":

            return json.dumps(

                {

                    "artifact_type": "memory_inconsistency_report",

                    "source_role": "anti_memory_probe",

                    "attack_vector": "memory_contradiction_probe",

                    "attacked_role": "critic",

                    "failure_score": 0.40,

                    "reproducibility": 0.95,

                    "summary": "Role-local memory contains a minor phrasing mismatch,
but it is low impact.",
```

```
            "evidence": ["Mismatch does not materially change the task."],
            "promotion": {
                "kind": "constraint",
                "title": "Minor memory cleanup",
                "content": "Normalize phrasing before reuse.",
                "tags": ["memory"],
                "metadata": {},
            },
            "memory_keys": ["planner", "critic"],
            "contradiction": "The critic summary is shorter than the planner
trace.",
            "proposed_resolution": "Normalize summary verbosity before storing
it.",
        }
```

```
            )

        if role == "anti_tool_stress":

            return json.dumps(

                {

                    "artifact_type": "tool_fuzz_case",

                    "source_role": "anti_tool_stress",

                    "attack_vector": "tool_boundary_fuzz",

                    "attacked_role": "tool_router",

                    "failure_score": 0.81,

                    "reproducibility": 0.74,

                    "summary": "A malformed simulated tool input exposes a brittle
boundary in task handoff handling.",

                    "evidence": ["The payload mixes expected and unexpected fields."],
```

```
                    "promotion": {
                        "kind": "test",
                        "title": "Tool handoff fuzz case",
                        "content": "Validate tool payloads with mixed optional and
malformed fields.",
                        "tags": ["tool", "fuzz"],
                        "metadata": {"source": "stub"},
                    },
                    "tool_name": "simulated_tool",
                    "crafted_input": {"mode": "execute", "unexpected": ["edge-case"]},
                    "expected_failure_mode": "Boundary validation passes malformed
payloads downstream.",
                }
            )
```

```
        if role == "anti_executor":

            return json.dumps(

                {

                    "artifact_type": "counterexample_task_variant",

                    "source_role": "anti_executor",

                    "attack_vector": "counterexample_variant",

                    "attacked_role": "executor",

                    "failure_score": 0.78,

                    "reproducibility": 0.82,

                    "summary": "A counterexample variant stresses the plan with
conflicting deadlines and incomplete context.",

                    "evidence": ["Variant removes the key supporting assumption from
the prompt."],

                    "promotion": {
```

```
                    "kind": "curriculum",

                    "title": "Counterexample prompt variant",

                    "content": "Practice the same task with incomplete context and
conflicting deadlines.",

                    "tags": ["curriculum"],

                    "metadata": {"source": "stub"},

                },

                "variant_task": "Solve the task again with incomplete context and a
conflicting deadline.",

                "expected_failure_mode": "The executor overfits to the original
clean scenario.",

            }

        )

    if role == "anti_retrieval_probe":
```

```
            return json.dumps(
                {
                    "artifact_type": "retrieval_poison_probe",
                    "source_role": "anti_retrieval_probe",
                    "attack_vector": "retrieval_poison_probe",
                    "attacked_role": "planner",
                    "failure_score": 0.73,
                    "reproducibility": 0.67,
                    "summary": "A plausible lure document reframes the task objective
and shifts the planner off course.",
                    "evidence": ["Planner prioritizes the lure over the explicit task
objective."],
                    "promotion": {
                        "kind": "constraint",
```

```
                        "title": "Objective anchoring guard",

                        "content": "Require the planner to restate the objective before
using retrieved context.",

                        "tags": ["retrieval"],

                        "metadata": {"source": "stub"},

                    },

                    "query": "task objective",

                    "lure_docs": ["A plausible but misleading document that flips the
objective priority."],

                    "why_it_misleads": "It sounds authoritative and is semantically
close to the original task.",

                }

            )

        return f"Unhandled role: {role}"
```

```
class DeterministicRuntime(OmegaRuntime):

    def _select_strategies(self, *, count: int):

        return self._default_strategy_catalog()[:count]
```

```
class GeometryTests(unittest.TestCase):

    def test_calculate_geometry_basic(self):

        history = {"overlap": [1.0, 0.9, 0.8], "mu_ema": [0.0, 0.1, 0.2]}

        result = calculate_geometry(history)
```

```
        self.assertIn("phi", result)

        self.assertIn("mu", result)

        self.assertIn("jerk", result)

        self.assertIn("theta", result)

        self.assertTrue(np.isclose(result["phi"], 0.9))

        self.assertTrue(np.isclose(result["mu"], 0.1))

        self.assertGreaterEqual(result["theta"], 0.0)




    def test_calculate_geometry_clipping(self):

        history = {"overlap": [1.5], "mu_ema": [-0.5]}

        result = calculate_geometry(history)
```

```
        self.assertTrue(np.isclose(result["theta"], 0.0))




class FrameworkModelTests(unittest.TestCase):

    def test_parse_rejects_malformed_artifact(self):

        malformed = {

            "artifact_type": "plan_trap",

            "source_role": "anti_planner",

            "attack_vector": "trap",

            "failure_score": 1.5,

            "reproducibility": 0.9,

            "summary": "Too short",
```

```
            "plan_delta": "delta",

            "hidden_assumption": "hidden",

            "expected_failure_mode": "failure",

        }

        with self.assertRaises(Exception):

            parse_anti_artifact(malformed)




def test_parse_accepts_sanitized_plan_trap(self):

        artifact = parse_anti_artifact(

            {

                "artifact_type": "plan_trap",

                "source_role": "anti_planner",
```

```
                "attack_vector": "hidden_assumption_trap",

                "attacked_role": "planner",

                "failure_score": 0.88,

                "reproducibility": 0.83,

                "summary": "This plan trap exposes a hidden assumption in the
constructive plan.",

                "evidence": ["assumption mismatch"],

                "promotion": {

                    "kind": "test",

                    "title": "Assumption trap",

                    "content": "Inject a conflict and verify the planner reconciles
it.",

                    "tags": ["planner"],

                    "metadata": {"source": "unit"},
```

```
                },

                "plan_delta": "Force a contradictory constraint into step two.",

                "hidden_assumption": "All constraints remain compatible.",

                "expected_failure_mode": "Planner ignores the contradiction and
produces a brittle plan.",

            }

        )

        self.assertIsInstance(artifact, PlanTrap)

        self.assertEqual(artifact.promotion.kind, "test")









class FrameworkMemoryTests(unittest.TestCase):
```

```
    def test_memory_boundaries(self):

        with tempfile.TemporaryDirectory() as tmp_dir:

            base = Path(tmp_dir)

            role_memory = RoleLocalMemory("planner", base_dir=base / "role_local")

            anti_memory = AntiPrivateMemory(base_dir=base / "anti_private")

            record = MemoryRecord(source="planner", content="planner note",
metadata={})




def test_calculate_geometry_clipping():

    """Ensure acos clipping handles invalid float maths gracefully."""

    history = {

        'overlap': [1.5], # Should clip to 1.0 inside theta calc

        'mu_ema': [-0.5]
```

```
    }

    result = calculate_geometry(history)

    assert np.isclose(result["theta"], 0.0) # arccos(1.0) == 0.0
```

```
            role_memory.append(record, writer_role="planner")

            self.assertEqual(len(role_memory.read(reader_role="planner")), 1)

            self.assertEqual(len(role_memory.read(reader_role="anti",
reader_manifold="anti")), 1)
```

```
            anti_memory.append(MemoryRecord(source="anti_planner", content="sealed
note", metadata={}))

            self.assertEqual(len(anti_memory.read(reader_manifold="anti")), 1)

            with self.assertRaises(PermissionError):

                anti_memory.read(reader_manifold="zero")
```

```
def test_shared_consensus_rejects_non_governor_write(self):

    tmp_dir = Path(tempfile.mkdtemp())

    try:

        scm = SharedConsensusMemory(db_path=tmp_dir / "memories.db")

        promotion = SanitizedPromotion(

            kind="test",

            title="Guardrail",

            content="Exercise the failure mode in a safe harness.",

            tags=["unit"],

        )

        with self.assertRaises(PermissionError):
```

```
            scm.promote(promotion, source="anti_planner", writer="anti")

        del scm

    finally:

        db_path = tmp_dir / "memories.db"

        if db_path.exists():

            try:

                os.remove(db_path)

            except OSError:

                pass

        shutil.rmtree(tmp_dir, ignore_errors=True)
```

```
class FrameworkGovernorTests(unittest.TestCase):

    def test_regime_classification_and_promotion_gate(self):

        governor = OmegaGovernor()

        baseline = governor.measure_trace(["steady plan", "steady plan with slight update"], action_count=2, memory_churn=1)

        artifact = MemoryInconsistencyReport(

            source_role="anti_memory_probe",

            attack_vector="memory_probe",

            attacked_role="critic",

            failure_score=0.90,

            reproducibility=0.90,

            summary="A reproducible memory contradiction degrades the constructive manifold.",

            evidence=["critique diverges from stored planner assumption"],
```

```
            promotion=SanitizedPromotion(
                kind="constraint",
                title="Memory consistency guard",
                content="Reconcile conflicting memory assumptions before critique
reuse.",
                tags=["memory"],
            ),
            memory_keys=["planner", "critic"],
            contradiction="Planner and critic disagree on a governing assumption.",
            proposed_resolution="Force a consistency check before critique reuse.",
        )
        decision = governor.score_attack(
            baseline,
```

```
            artifact,

            ["steady plan", "divergent execution", artifact.summary],

            action_count=4,

            memory_churn=4,

        )

        self.assertTrue(decision.promote)

        self.assertIn(decision.metrics.regime, {Regime.FLOW, Regime.VISCOSITY})




        unsafe = governor.score_attack(

            baseline,

            artifact.model_copy(update={"failure_score": 0.95}),

            ["completely unrelated text", "chaotic drift", "another disconnected
branch", artifact.summary],
```

```
            action_count=8,

            memory_churn=7,

        )

        self.assertFalse(unsafe.promote)

        self.assertEqual(unsafe.metrics.regime, Regime.TURBULENCE)









class FrameworkRuntimeTests(unittest.IsolatedAsyncioTestCase):

    async def test_evolve_task_promotes_and_quarantines(self):

        base = Path(tempfile.mkdtemp())

        try:

            runtime = DeterministicRuntime(
```

```
                backend=StubBackend(),

                scm=SharedConsensusMemory(db_path=base / "memories.db"),

                role_memory_dir=base / "role_local",

                anti_memory_dir=base / "anti_private",

                random_seed=7,

            )

            result = await evolve_task(

                TaskEnvelope(objective="Design a bounded deployment plan for a risky
system upgrade."),

                runtime=runtime,

            )



            self.assertEqual(result.constructive.trace[0].role, "planner")
```

```
            self.assertGreaterEqual(len(result.attack_reviews), 1)

            self.assertGreaterEqual(len(result.promoted_entries), 1)

            self.assertIn(result.final_regime, {Regime.FLOW, Regime.VISCOSITY,
Regime.TURBULENCE})







            scm_records = runtime.scm.read(limit=10)

            self.assertGreaterEqual(len(scm_records), 1)

            apm_records = runtime.apm.read(reader_manifold="anti", limit=10)

            self.assertEqual(len(apm_records), len(result.attack_reviews))

            del runtime

        finally:

            db_path = base / "memories.db"

            if db_path.exists():
```

```
            try:

                os.remove(db_path)

            except OSError:

                pass

        shutil.rmtree(base, ignore_errors=True)




async def test_legacy_imports_and_serc_bridge_stay_available(self):

    from agent_zero import Agent, SERC, evolve_task as exported_evolve_task




    self.assertTrue(callable(exported_evolve_task))

    self.assertTrue(hasattr(Agent, "reason"))

    self.assertTrue(hasattr(SERC, "run_dual_manifold_cycle"))
```

```
if __name__ == "__main__":

    unittest.main()
```

You’re out of Codex messages Your rate limit resets on Apr 21, 2026, 8:04 PM. To continue using Codex, upgrade to Plus today.

Upgrade

GPT-5.4

Extra High

Local 0%

master

Default